

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

**Практическая работа “ЛР3: Контейнеризация написанного приложения
средствами Docker”**

Выполнил:
Березина Софья
Группа
К3339

Проверил: Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Целью лабораторной работы являлась контейнеризация микросервисной версии fitness-platform средствами Docker. Требовалось подготовить Dockerfile для каждого сервиса, описать общий docker-compose.yml и настроить сетевое взаимодействие между контейнерами приложения.

Ход работы

Для каждого микросервиса был создан отдельный Dockerfile. Все сервисы используют базовый образ node:18-alpine, рабочую директорию /app, копирование package.json, установку production-зависимостей через npm install --omit=dev, копирование исходного кода и запуск команды npm start. Для HTTP-сервисов объявлен порт 3000 внутри контейнера.

Общий файл docker-compose.yml находится в каталоге labs/lab3. В нем описаны сервисы auth, user, workout, userworkouts, progress и blog, а также инфраструктурные контейнеры RabbitMQ и шесть отдельных PostgreSQL-баз данных. Для каждого приложения заданы переменные окружения: PORT, DATABASE_URL и, где необходимо, RABBITMQ_URL и URL зависимых сервисов.

Для внешнего доступа микросервисы проброшены на разные порты хоста: auth доступен на 3001, user на 3002, workout на 3003, userworkouts на 3004, progress на 3005, blog на 3006. RabbitMQ доступен на портах 5672 и 15672. Внутри Docker Compose сервисы обращаются друг к другу по DNS-именам контейнеров, например http://user:3000 и http://workout:3000.

Для сохранения состояния настроены Docker volumes: rabbitmq_data, auth_db_data, users_db_data, workouts_db_data, user_workouts_db_data, progress_db_data и blog_db_data. Благодаря этому данные PostgreSQL и RabbitMQ не теряются при обычном перезапуске контейнеров. При необходимости полностью очистить окружение можно выполнить docker compose down -v.

Сетевое взаимодействие между сервисами обеспечивается стандартной bridge-сетью Docker Compose. Порядок запуска задан через depends_on, а сами приложения дополнительно реализуют ожидание готовности БД и RabbitMQ с повторными попытками подключения. Это делает запуск командой docker compose up --build устойчивым к тому, что база данных или брокер сообщений стартуют медленнее приложения.

Вывод

В результате лабораторной работы микросервисная версия fitness-platform была полностью контейнеризирована. Для каждого сервиса подготовлен Dockerfile, общий docker-compose.yml поднимает приложения, базы данных и RabbitMQ, а сетевое взаимодействие настроено через внутреннюю сеть Docker Compose и переменные окружения.